

AI-ARKITEKTUR & FÖRSTÅELSE

AI-Basics

En liten guide till modeller, gränssnitt, agenter och sånt där som vi hör så mycket om numera...

DEL 1/2

Kärnan: Motor, Gränssnitt &

Delarna och hur de hänger ihop

Agent

Systemets tre huvudkomponenter

1. MOTORN / BERÄKNING

Modell / LLM

Motorn som bearbetar instruktioner och genererar svar (t.ex. Claude, GPT-4o, Llama). Den är i grunden *tillståndslös* (stateless).

Minne, VRAM & Resurs: Kräver VRAM (GPU / Mac Unified Memory) för att köras. Parametrar och kvantisering avgör storleken, medan KV-cache äter dynamiskt minne vid lång kontext.

Roll: Beräknar svar, men saknar eget minne och flåtkomst utan hjälp.

2. PRESENTATIONSLAGRET

Chat / Editor / CLI

Ytan där människan formulerar mål och ser resultat (t.ex. terminalen, VS Code-panelen eller webbchatten).

Interaktionskanal: Förbrukar inga tunga beräkningsresurser i sig – förmedlar bara in- och utdata till användaren.

Roll: Samma modell upplevs helt annorlunda i ett IDE än i en webbchatt.

3. ORKESTRERINGEN

Agent / Harness

Programloopen bakom kulisserna som läser projektfiler, planerar delsteg, anropar verktyg och hanterar behörigheter.

Kontextkontroll: Avgör vad som skickas till motorn och hanterar verktygsscheman och filer.

Roll: Förvandlar text till faktisk handling genom att arbeta i steg (läs, ändra, testa) tills jobbet är klart.

I praktiken: CLI och Agent paketeras ofta tillsammans i samma verktyg, men skiljer sig åt i funktion.

Kontextfönstret: Arbetsminne & Gränser

Tokens & Arbetsminnets dynamik

Tokens som valuta: Text bryts ner i tokens (~4 tecken/token på engelska). Allt – systemprompt, filer, historik och resonemang (CoT) – delar på samma ändliga fönster (dvs det finns en gräns för arbetsminnet).

Relevans framför volym: Större fönster betyder inte automatiskt bättre svar. Överflödiga data skapar brus, ökar latens och minskar modellens skärpa.

Flyktigt tillstånd: Modellen glömmer allt mellan sessioner. Varje nytt anrop måste förses med rätt information för att fungera.

"Dumb Zone", Kompression & Persistens

Degradering vid mättnad: När kontexten sväller tappar modellen förmågan att följa strikta regler ("lost in the middle") och börjar istället hallucinera.

Kompression (Compaction / Pruning): När fönstret fylls kan agent-harneset rensa gamla verktygsutdata och sammanfatta delsteg för att återfå precision.

Persistens (Långtidsminne): Kritiska regler och arkitektur måste sparas permanent i filer (AGENTS.md) eller databas för att överleva kontextrensningar.

Prompts

Svag / Otydlig prompt

```
"Fixa buggen i inloggningen och gör koden snyggare."
```

Saknar kontext & filer: Modellen måste gissa var felet ligger och riskerar att läsa in fel kod.

Inga avgränsningar: "Gör snyggare" inbjuder till oönskad refaktorisering och regressioner.

Oklart mål: Saknar felmeddelande, förväntat beteende och verifieringskriterier.

Skarp / Målmedveten prompt

```
"Felsök felet 401 i auth_service.py:42. Ändra inte databasmodellen. Kör tests/test_auth.py och visa diffen innan ändring sparas."
```

Tydligt mål & filkontext: Pekar ut exakt rad, felkod och relevant testsvit.

Tydlig avgränsning: Sätter ett strikt förbud mot att ändra databasschemat.

Verifiering & format: Kräver testkörning och granskningsbar diff före exekvering.

Token-ekonomi & Gränssnittsstrategi

Webbchatt vs. CLI / IDE-agenter

Webbchatt (Fast månadspris): Bäst för idébollning, resonemang, arkitektur och textförfattande. Du kan föra långa dialoger utan rörlig kostnad per token.

CLI / IDE / API (Pay-per-token): Varje interaktion skickar om hela historiken, systemprompter och inlästa filer. Bör användas kirurgiskt för exakt kodarbete.

Kombinera rätt: Bolla arkitektur i webbchatt → implementera målinriktat via terminal/IDE-agent. Eller be webbchatt göra en tydlig prompt för att spara kontext i cli.

Tre principer för token-hygien

1. Peka istället för att klistra in: Ge agenten filpekare och radnummer (src/auth.ts:45–88) istället för att dumpa tusentals rader i prompten.

2. Färska sessioner (Fresh Sessions): Starta en ny session så fort ett delmål är löst för att hålla kontexten ren.

3. Model Tiering: Använd snabba/billigare modeller (t.ex. Haiku/Flash) för enkla skript och spara flaggskeppsmodeller för komplex problemlösning.

Kärnsikt: Token-ekonomi handlar inte bara om pengar – färre onödiga tokens ger snabbare svar och minskar risken för *Lost in the Middle*.

Dataintegritet, Träning & Säkerhet

Träning på data

Gratiskonton / Standardinställning (Webb): Data och prompter kan användas för att träna framtida modeller om man inte aktivt väljer bort det (opt-out).

Betalda Enterprise / API / Team-konton: Koderna och prompterna sparas inte för modellträning (Zero Data Retention / No Training-policy hos OpenAI, Anthropic m.fl.).

Känslig data i kodbasen

Hemligheter & Nycklar: API-nycklar, tokens och .env-filer ska alltid ignoreras (t.ex. via .cursorignore, .gitignore eller harness-filter).

Personuppgifter & Kunddata: Skicka aldrig riktig produktionsdata eller identifierbara personuppgifter i prompter eller felmeddelanden.

Företagshemligheter: Säkerställ att teamet känner till företagets AI-policy innan interna kärnalgorithmerna analyseras.

Agentens behörigheter

Körning av skript & verktyg: Var restriktiv med att ge agenter oövervakad terminalaccess för kommandon som gör ändringar i databaser, tar bort filer eller pushar kod direkt till main.

DEL 2 / 2

Varaktig struktur & utökad

Hur vi bygger stabila arbetsmetoder med AGENTS.md, Skills, MCP och RAG.

förmåga

Instruktioner & integrationer

VARAKTIGA REGLER

Agents

Projektets varaktiga regler och guide till kodagenter.

Beskriver projektmiljö, kodstil, testrutiner, terminalkommandon och säkerhetsregler.

ARBETSMÖNSTER

Skills

Återanvändbara instruktioner för specifika arbetsmoment.

Exempel: "grilla en idé", skriva teknisk specifikation, TDD-implementering eller kodgranskning.

EXTERNA KOPPLINGAR

MCP-protokollet

Öppen standard (Model Context Protocol) för att ansluta externa verktyg.

Ger åtkomst till databaser och API:er. Kräver medveten hantering av behörigheter.

Fördjupning: AGENTS.md

Syfte & Exempel på innehåll

Projektets grundlag: En Markdown-fil i repot som agenten läser in automatiskt vid start för att förstå projektets förutsättningar.

```
# Regler för agenten i projektet:  
• Stack: Node v20, Next.js App Router, Vitest  
• Test: Kör alltid `npm test` före slutförande  
• No-go: Modifiera aldrig db/schema.prisma direkt  
• Format: Presentera ändringar som kort git diff
```

Slipper upprepning: Utvecklaren behöver inte förklara projektets miljö och regler i varje ny chatt/session.

Fördelar & Bästa praxis

Fördel – Persistens & Teampraxis: Hela teamet delar samma guardrails för alla agenter. Versioneras direkt i git.

Fördel – Förhindrar regressioner: Stoppar agenten från att föreslå förlegade mönster eller köra felaktiga byggkommandon.

Fallgrop – "Context bloat": Skriv inte en roman. En alltför stor fil stjälar värdefulla tokens och sänker modellens skärpa.

Underhåll: Måste uppdateras kontinuerligt när beroenden eller arkitektur förändras.

Fördjupning: Skills & Arbetsflöden

Vad är en Skill & Typiska exempel

Modulära arbetsmönster: Färdiga promptmallar och processteg som laddas in för specifika, avgränsade uppgifter.

1. **"Grilla en idé" (Red Teaming):** Tvingar modellen att agera kritisk granskare som letar logiska brister och dolda risker.
2. **Spec-driven utveckling:** Bryter ner en feature i user stories, teknisk spec och acceptanskriterier innan kod skrivs.
3. **TDD-loop:** Guidad cykel: Skriv failande test → generera minimal kod → verifiera → refaktorisera.

För- & Nackdelar i vardagen

Fördel – Hög repeterbarhet: Standardiserar kvalitetsnivån i teamet oavsett vem som kör prompten.

Fördel – Mentalt stöd: Hjälper utvecklaren/agenten att hålla disciplinen (t.ex. att inte koda innan kravbilden är tydlig).

Begränsning – Inte magi: En skill kan inte ersätta mänskligt omdöme; modellen kan fortfarande göra logiska tankevirpor.

Risk för process-overhead: För många eller för rigida skills kan göra enkla uppgifter onödigt långsamma och dyra.

Fördjupning: MCP (Model Context Protocol)

Protokoll & Integrationer

Öppen standard: Ett enhetligt protokoll (initierat av bl.a. Anthropic) för att koppla AI-klienter till lokala och externa verktyg.

Databas-MCP: Låter agenten köra -frågor mot t ex PostgreSQL för att inspektera scheman och testdata.

GitHub / GitLab-MCP: Skapa PRs, läsa issues och hämta CI/CD-loggar direkt i agentens kontext.

API & Docs-MCP: Slå upp aktuell dokumentation för interna tjänster i realtid.

Möjligheter vs. Risker & Behörigheter

Fördel – Standardiserat ekosystem: Bygg en MCP-server en gång – använd den i alla kompatibla klienter och verktyg.

Risk – Behörigheter & Säkerhet: Att ge skrivrättigheter till en databas eller produktions-API innebär att modellen kan radera data.

Kontextkostnad: Varje ansluten MCP-server definierar verktygsscheman i prompten, vilket förbrukar värdefulla tokens.

Ingen automatisk QA: MCP ger åtkomst, men garanterar inte att modellens tolkning eller SQL-fråga är felfri.

RAG: App-arkitektur vs. Agentverktyg

RAG: Mjukvaruarkitektur i appar

Passiv datahämtning i backend: Ett arkitekturmönster för att bygga AI-funktioner i applikationer (t.ex. kundtjänstbotar mot 10 000 dokument).

Hur det fungerar: Appens backend gör en semantisk sökning i en databas och klistrar in relevanta textutdrag i prompten *innan* modellen anropas.

Roll i systemet: Ingen autonomi krävs av modellen – systemets mjukvara styr exakt vilken data som injiceras i kontexten.

Agent / MCP: Interaktiv verktygskörning

Aktiv handling i ett harness: Utvecklarens interaktiva miljö där agenten har egna verktyg för att utföra uppgifter.

Hur det skiljer sig: Modellen bestämmer *själv under körning* när den behöver läsa en specifik fil, köra en SQL-fråga eller anropa ett MCP-verktyg.

Nyckelinsikt för utvecklare: RAG är något du bygger *inuti en produkt* – inte ett vardagligt krav för att koda assisterat med agenter och MCP.

Sammanfattning: RAG är ett *applikationsmönster* för extern data. Agent-harnesses använder istället *aktiv verktygskörning & filsystemsåtkomst*.

Sammanfattning & Frågor

Hur kan vi använda denna mentala modell för att fatta bättre beslut kring prompter, verktyg och agentstyrning i vardagen?